GitHub Candidate Agent — Complete Project Deep Dive

> *A fully automated pipeline that discovers, analyzes, scores, and ranks Indian student developers on GitHub for recruitment purposes, with a React dashboard for exploring results.*

🏗️ Architecture Overview

```
graph TB
    subgraph "Data Pipeline (main.py)"
        A["1. Search GitHub<br/>stages/search.py"] ⟶ B["2. Filter Candidates<br/>s
        B ⟶ C["3. Extract Links<br/>stages/link_extractor.py"]
        C ⟶ D["4. Analyze Activity<br/>stages/activity_analyzer.py"]
        D ⟶ E["5. Analyze Projects<br/>stages/project_analyzer.py"]
        E ⟶ F["6. Categorize Domain<br/>stages/categorizer.py"]
        F ⟶ G["7. Score Candidate<br/>stages/scorer.py"]
        G ⟶ H["8. Save & Export<br/>checkpoint.py / sheets_writer.py"]
        H ⟶ I["9. Email Notification<br/>notifier.py"]
    end

    subgraph "Hybrid Pipeline (pipeline.py)"
        J["Fetch Profile"] ⟶ K["PARALLEL"]
        K ⟶ K1["Embeddings<br/>embeddings.py"]
        K ⟶ K2["LLM Classify<br/>llm_classifier.py"]
        K ⟶ K3["LLM Links<br/>llm_classifier.py"]
        K ⟶ K4["LLM Score<br/>llm_classifier.py"]
        K1 & K2 & K3 & K4 ⟶ L["ML Rank<br/>feedback.py"]
        L ⟶ M["Merge → Final Score"]
        M ⟶ N["Save to SQLite<br/>db.py"]
    end
```

```
    subgraph "Backend API (backend/api.py)"
        O["FastAPI Server<br/>Port 8000"]
        O ⟶ P["/candidates"]
        O ⟶ Q["/candidates/by-date"]
        O ⟶ R["/candidates/{login}/enrich"]
        O ⟶ S["/ws/run-pipeline"]
        O ⟶ T["/train"]
        O ⟶ U["/role-fit"]
        O ⟶ V["/duplicates"]
    end

    subgraph "Frontend (frontend/)"
        W["React + Vite<br/>Port 5173"]
        W ⟶ X["Dashboard"]
        W ⟶ Y["Candidate Details"]
        W ⟶ Z["Enrichment View"]
        W ⟶ AA["ML Training"]
        W ⟶ AB["Role Fit"]
        W ⟶ AC["CEO Profile"]
        W ⟶ AD["Duplicate Detector"]
        W ⟶ AE["Compare View"]
    end

    H -•->|JSON cache| O
    O ⟷|REST API| W
```

📂 File-by-File Breakdown

Root-Level Configuration

File	Purpose
config.py	Central configuration: loads `.env`, defines scoring weights, Indian/student keywords, domain→language mappings

File	Purpose
.env	Secrets: `GITHUB_TOKEN`, `HF_TOKEN`, `GOOGLE_SHEET_ID`, `EMAIL_*`, `GEMINI_API_KEY`
requirements.txt	Python dependencies (FastAPI, PyGithub, XGBoost, sentence-transformers, etc.)

Root-Level Core Files

File	Purpose
database.py	Simple SQLite — `processed_profiles` table tracking which logins were already analyzed
db.py	Hybrid pipeline's SQLite — `candidates` table storing full profile JSON, embeddings, and human labels
checkpoint.py	JSON-file cache (`candidates_cache.json`) — stores all enriched candidate data between runs
notifier.py	Email notification via Gmail SMTP — sends HTML reports with CSV attachments after each pipeline run
embeddings.py	Sentence-transformer semantic matching — classifies bios as student/job_seeker/developer
llm_classifier.py	Gemini LLM integration — classification, scoring, and link extraction using `gemini-3.1-pro-preview`
feedback.py	ML feedback loop — XGBoost classifier trained on human-labeled candidates (good/bad)
scheduler.py	Runs the pipeline automatically every Monday at 06:00

🔄 Pipeline 1: The Original Pipeline (`main.py`)

Entry point: `python main.py`

This is the **batch discovery** pipeline that searches GitHub for Indian student developers.

Stage-by-Stage Flow

Stage 1 — Search (stages/search.py)

```
Function: fetch_candidate_logins(max_per_query=50)
```

- Runs **6 parallel GitHub search queries** targeting Indian students:
- `location:India "student" in:bio language:Python`
- `location:India "btech" in:bio`
- `location:India "undergrad" in:bio`
- `location:India "internship" in:bio followers:>5`
- `location:India "open to work" in:bio`
- `location:India "iit" OR "nit" OR "bits" in:bio`
- Uses `ThreadPoolExecutor` to run all queries concurrently
- Deduplicates by login name
- Returns a list of GitHub usernames (strings)

Stage 2 — Filter (stages/filters.py)

```
Function: filter_candidates(logins) → list[NamedUser]
```

Two filter functions applied to each candidate:

`is_indian(user)` - Checks `location`, `bio`, `company`, `name` against `config.INDIAN_KEYWORDS` - Keywords include: Indian cities (bangalore, mumbai, delhi...), states, and colleges (iit, nit, bits)

`is_student(user)` - Checks `bio` and `name` against `config.STUDENT_KEYWORDS` - Keywords include: student, btech, undergrad, looking for internship, etc.

Only users passing **both** filters proceed. Each login is fetched via the GitHub API (`get_user`) to get full profile data.

Stage 3 — Link Extraction (stages/link_extractor.py)

```
Function: extract_links(user) → list[dict]
```

Extracts URLs from three sources: 1. **Blog/website field** — the GitHub profile's blog URL 2. **Bio text** — regex-scans for embedded URLs 3. **Email** — extracted as `mailto:` link

Each URL is classified against known domains: - `linkedin.com` → "linkedin" - `twitter.com` / `x.com` → "twitter" - `leetcode.com`, `kaggle.com`, `medium.com`, etc. - Anything else → "website"

Stage 4 — Activity Analysis (stages/activity_analyzer.py)

```
Function: analyze_activity(user) → dict
```

Scans **all repos** and their commits within the last `COMMITS_LOOKBACK` (90) days:

Output Key	Description
`total_commits`	Total commits in lookback window
`consistency_score`	`active_days / 90` — fraction of days with commits (0–1)
`top_languages`	Top 8 languages sorted by bytes written across all repos
`stars_total`	Sum of stars across all repos
`commit_quality_ratio`	Fraction of commits with meaningful messages (not "fix", "update", "wip", etc.)

Output Key	Description
`garbage_commits`	Count of low-quality commit messages
`active_days`	Number of distinct dates with commits

Commit quality detection: Uses regex to flag generic messages like "initial commit", "update", "fix", "wip", "merge branch", etc.

Stage 5 — Project Analysis (stages/project_analyzer.py)

```
Function: analyze_projects(user) → list[dict]
```

Analyzes the **top 5 non-fork repos** (by stars): 1. Reads the README (or falls back to description) 2. Uses **HuggingFace zero-shot classification** (`facebook/bart-large-mnli`) to classify: - **Project quality**: "original project" vs "tutorial clone" vs "complex system" vs "simple script" - **Complexity**: "beginner" vs "intermediate" vs "advanced" 3. Extracts topics and languages 4. Returns `is_original` (true if quality score > 0.4), `complexity` , and `quality_score`

Stage 6 — Domain Categorization (stages/categorizer.py)

```
Function: categorize(activity, projects) → dict
```

Two-pass approach:

1. **Rule-based pass**: Matches the candidate's top languages against `config.DOMAIN_LANGUAGES` :
2. JavaScript/TypeScript → frontend
3. Python/Java/Go → backend
4. Jupyter Notebook → ai_ml
5. Dockerfile/Shell → devops
6. Swift/Kotlin/Dart → mobile

7. **AI refinement**: Uses HuggingFace zero-shot classification on project descriptions to blend AI scores with the rule-based scores

Returns `primary_domain` and per-domain confidence scores.

Stage 7 — Scoring (stages/scorer.py)

```
Function: score_candidate(activity, projects, category) → dict
```

Computes a **weighted final score (0–1)**:

Component	Weight	How It's Calculated
`project_quality`	**35%**	Average `quality_score` of original repos
`consistency`	**25%**	`consistency_score` from activity analysis
`tech_depth`	**20%**	`n_languages × 0.1 + advanced_projects × 0.2` (capped at 1.0)
`activity`	**20%**	`commit_volume × 0.4 + star_volume × 0.3 + commit_quality × 0.3`

Also generates human-readable **pros** and **cons**: - Pros: "Highly consistent committer", "High-quality original projects", etc. - Cons: "Low commit consistency", "Mostly tutorial/homework repos", etc.

Stage 8 — Export (stages/sheets_writer.py + checkpoint.py)

- Saves each candidate to `candidates_cache.json` via checkpoint system
- Records in SQLite `processed_profiles` table (for deduplication)
- Writes top N candidates to a **Google Sheet** (new tab per date)
- Falls back to CSV export if Google Sheets fails

Stage 9 — Notification (notifier.py)

- Sends an HTML email report via **Gmail SMTP** with:
- Summary statistics (new, skipped, total)
- Top 10 candidates in an HTML table
- CSV attachment with full candidate data

🔄 Pipeline 2: The Hybrid Pipeline (`pipeline.py`)

Entry point: `python pipeline.py <github_username>`

This is an **alternative, per-user pipeline** that uses LLMs and ML for deeper analysis.

How It Differs from Pipeline 1

Aspect	Pipeline 1 (`main.py`)	Pipeline 2 (`pipeline.py`)
Mode	Batch (discovers hundreds)	Single user (deep analysis)
AI	HuggingFace zero-shot	Gemini LLM + sentence-transformers
Scoring	Rule-based weights	LLM scoring + ML classifier
Storage	JSON cache + Google Sheets	SQLite with embeddings
Speed	~1-2 min per user	~7-10s per user (parallel)

Hybrid Pipeline Stages

```
Step 1: Fetch GitHub profile (parallel repo details)
         │
Step 2-5 run IN PARALLEL:
         ├── Embeddings (sentence-transformers/all-MiniLM-L6-v2)
         │   → student_score, job_seeker_score, developer_score
         ├── LLM Classification (Gemini 3.1 Pro)
```

```
          │   → is_indian, is_student, is_job_seeker + confidence
          ├── LLM Link Extraction (Gemini)
          │   → linkedin, portfolio, resume, twitter, github_pages
          └── LLM Scoring (Gemini)
              → total_score/100, strengths, weaknesses, recruiter_summary
          │
Step 6:  ML Ranking (XGBoost, if trained model exists)
         → probability of "good" candidate
          │
Step 7:  Merge all signals → final_score
         Formula: 0.5×LLM + 0.3×ML + 0.2×Embedding (or 0.7×LLM + 0.3×Embedding if no
          │
Step 8:  Save to hybrid_candidates.db
```

Key Components

embeddings.py — Semantic Matching

- Model: `sentence-transformers/all-MiniLM-L6-v2` (384-dim vectors)
- Compares bio text against reference phrases for "student", "job_seeker", "developer"
- Uses cosine similarity to compute scores
- Threshold: 0.35 — below this returns "unclear"

llm_classifier.py — Gemini LLM

- Model: `gemini-3.1-pro-preview`
- **Task 1 — Classification**: Is this person Indian? Student? Job seeker? (with confidence scores)
- **Task 3 — Scoring**: Score out of 100 with 5-category breakdown + recruiter summary
- **Task 4 — Link Extraction**: Hybrid regex + LLM to find LinkedIn, portfolio, resume, etc.
- All outputs validated as strict JSON

feedback.py — ML Feedback Loop

- Human labels candidates as "good" or "bad" via the dashboard

- Once 50+ labels exist, trains XGBoost (or LogisticRegression fallback)
- Features: `[llm_score, student_score, job_seeker_score, repo_count, commit_count]`
- Trained model saved to `candidate_classifier.pkl`
- Used at inference time to predict probability of "good"

🌐 Backend API (backend/api.py)

Framework: FastAPI | **Port:** 8000 | **1003 lines of code**

Core Endpoints

Endpoint	Method	Purpose
`/candidates`	GET	List all candidates with filtering (domain, min_score, has_linkedin, has_cv)
`/candidates/{login}`	GET	Get single candidate details
`/candidates/by-date?date=YYYY-MM-DD`	GET	Filter by processing date
`/candidates/by-week?week=YYYY-WNN`	GET	Filter by ISO week
`/stats`	GET	Aggregate stats (total, avg score, top score, domains)
`/dates`	GET	Available processing dates for date picker
`/weeks`	GET	Available weeks for week selector

Enrichment Endpoints (On-Demand, Cached)

These hit the GitHub API lazily and cache results in `enrichment_cache.json`:

Endpoint	Method	What It Analyzes
`/candidates/{login}/staleness`	GET	How recently active (🔥 Active Now → ❄️ Gone Cold)
`/candidates/{login}/commit-pattern`	GET	Night Owl / Weekend Warrior / Daily Coder archetype
`/candidates/{login}/code-quality`	GET	Multi-repo analysis: tests, tools, README, complexity grade
`/candidates/{login}/live-projects`	GET	Deployed projects (Vercel, Netlify, GitHub Pages, etc.)
`/candidates/{login}/enrich`	GET	Runs staleness + commit-pattern + live-projects in parallel
`/candidates/{login}/red-flags`	GET	Ghost developer, fork-only, tutorial-only, padded portfolio
`/candidates/{login}/collaboration`	GET	PRs to others, issues, stars, forks, org memberships
`/candidates/{login}/growth`	GET	Tech complexity trend, repo creation rate, stars trend
`/candidates/{login}/full-profile`	GET	Red flags + collaboration + growth in parallel

Action Endpoints

Endpoint	Method	Purpose
`/ws/run-pipeline`	WebSocket	Stream pipeline output live to the dashboard
`/send-report`	POST	Email HTML report with CSV for a date range
`/send-emails`	POST	Send personalized emails to selected candidates
`/candidates/export/csv`	GET	Download filtered candidates as CSV
`/candidates/{login}/label`	POST	Label candidate as good/bad for ML training
`/train`	POST	Trigger XGBoost training
`/model-status`	GET	Check if trained model exists
`/label-counts`	GET	Count of good/bad labels
`/role-fit`	POST	Score all candidates against a CEO-defined job role
`/duplicates`	GET	Detect duplicate candidates

🎨 Frontend (frontend/)

Framework: React 19 + Vite 8 | **Router:** react-router-dom | **Charts:** Recharts | **Icons:** lucide-react

Pages (8 total)

Route	Component	Purpose
`/`	Dashboard.jsx	Main view — stats cards, candidate table with filters
`/candidate/:login`	CandidateDetails.jsx	Detailed profile view
`/enrich/:login`	EnrichmentView.jsx	Deep enrichment: code quality, staleness, commit patterns, live projects
`/profile/:login`	CEOProfilePage.jsx	CEO intelligence: red flags, collaboration, growth trajectory
`/ml-training`	MLTraining.jsx	Label candidates good/bad, trigger model training
`/compare`	CompareView.jsx	Side-by-side comparison of candidates
`/role-fit`	RoleFitPage.jsx	Define job requirements, see match % for all candidates
`/duplicates`	DuplicateDetector.jsx	Find duplicate candidate profiles

Reusable Components (8 total)

Component	Purpose
CandidateTable	Sortable, filterable table of candidates
CandidateModal	Popup with candidate details
FilterPanel	Domain, score, LinkedIn/CV filters
WeekSelector	Date/week picker for time-based filtering

Component	Purpose
PipelineControl	Run pipeline via WebSocket, stream live output
ReportExporter	Send email reports for date ranges
EmailComposer	Compose and send personalized emails to candidates
SavedFilters	Save and reuse filter presets

🔌 Enrichment Stage Details

stages/staleness.py — Activity Recency

Uses `pushed_at` timestamps from repos (fast, no commit fetching):

Badge	Days Since Last Push
🔥 Active Now	0–7 days
✅ Recent	7–30 days
🌡️ Cooling	30–90 days
❄️ Gone Cold	90+ days

Also detects if the user is **actively job searching** (recent activity + bio keywords like "open to work").

stages/commit_pattern.py — Behavioral Archetype

Analyzes commit timestamps to classify:

Archetype	Detection Rule
Night Owl	40%+ commits between 10PM–5AM
Weekend Warrior	55%+ commits on Saturday/Sunday
Daily Coder	18+ distinct active days in 90 days
Sprinter	70%+ commits in any 14-day window
Balanced	No dominant pattern

Returns heatmap data for visualization.

stages/code_quality.py — Multi-Repo Analysis

Analyzes up to 20 non-fork repos in parallel: - **Tests**: % of repos with test files (conftest.py, jest.config, etc.) - **README coverage**: % of repos with README - **Tool detection**: 25+ patterns (Docker, GitHub Actions, Pytest, React, Next.js, Django, etc.) - **Python complexity**: Uses `radon` for cyclomatic complexity grading - **Language breakdown**: Aggregated bytes across all repos

Overall grade: A–F based on weighted scoring.

stages/red_flags.py — Warning Detection

Flag	Severity	Condition
👻 Ghost Developer	High	0 commits, 0 active days
🍴 Mostly Forks	Medium	85%+ repos are forks
📚 Tutorial-Only	Medium	70%+ repos match course patterns
🌫️ Inactive Account	Medium	Generic bio + <5 commits + 0 stars

Flag	Severity	Condition
⭐ No Traction	Low	0 stars across 5+ repos
🔵 Single Language	Low	Only 1 language with 10+ commits
💬 Poor Commit Hygiene	Low	50%+ garbage commit messages
🎭 Padded Portfolio	Medium	3+ repos with <4 files each
📄 No Documentation	Low	<20% repos have README

stages/collaboration.py — Community Presence

Scores 0–100 based on: - PRs to others' repos (5 pts each, max 30) - Issues opened (3 pts each, max 15) - Stars received (log-scaled, max 20) - Forks by others (log-scaled, max 15) - Followers (log-scaled, max 10) - Org memberships (2 pts each, max 10)

Labels: Team Player (70+) → Engaged (45+) → Solo Developer (20+) → Minimal Presence

stages/growth.py — Growth Trajectory

Compares older repos vs newer repos: - **Language complexity trend**: Are they using more advanced languages over time? - **Repo creation rate**: Accelerating or slowing? - **Stars trend**: Are newer repos getting more recognition? - **Tech expansion**: New languages in last 6 months

Trajectories: 📈 Growing Fast → 📈 Steady Growth → ➡️ Plateaued → 📉 Slowing Down

stages/role_fit.py — Job Match Scoring

CEO defines a job role with required/nice-to-have skills, thresholds, etc. Each candidate gets a **match %** based on: - Required skills (40%) — with alias normalization (e.g., "js" → "JavaScript") - Activity quality (20%) — pipeline score + consistency bonus - Code quality (20%) — from enrichment grade - Nice-to-have skills (10%) — partial credit - Thresholds (10%) — min commits, min stars, needs tests, domain match

🗄️ Data Storage

Store	File	Used By
Candidate cache	`candidates_cache.json`	Pipeline 1 → API → Frontend
Processing history	`candidates.db`	Deduplication, date/week filtering
Hybrid profiles	`hybrid_candidates.db`	Pipeline 2, ML training
Enrichment cache	`enrichment_cache.json`	On-demand enrichment (cached per-candidate)
ML model	`candidate_classifier.pkl`	XGBoost inference

🔑 External Services

Service	Purpose	Config Key
GitHub REST API	Fetch profiles, repos, commits	`GITHUB_TOKEN`
HuggingFace Inference	Zero-shot classification (bart-large-mnli) + summarization (bart-large-cnn)	`HF_TOKEN`
Google Gemini	LLM classification, scoring, link extraction	`GEMINI_API_KEY`
Google Sheets	Export results	`GOOGLE_SHEET_ID` + `credentials/service_account.json`

Service	Purpose	Config Key
Gmail SMTP	Email notifications	`EMAIL_SENDER` `EMAIL_PASSWORD` `EMAIL_RECEIVER`

🚀 How to Run

```
# 1. Activate virtual environment
cd c:\Users\jaisw\Desktop\github-candidate-agent
.\venv\Scripts\activate

# 2. Install dependencies
pip install -r requirements.txt

# 3. Start Backend API (Terminal 1)
cd backend
python -m uvicorn api:app --host 0.0.0.0 --port 8000 --reload

# 4. Start Frontend (Terminal 2)
cd frontend
npm install
npm run dev

# 5. Run Discovery Pipeline (Terminal 3, optional)
python main.py

# 6. Run Hybrid Pipeline on a specific user
python pipeline.py <github_username>
```

> *[!IMPORTANT] The backend MUST run on **port 8000** — all frontend components have this hardcoded.*